

Automatic Identification System (AIS)

Introduction to the Backend Architecture – Context

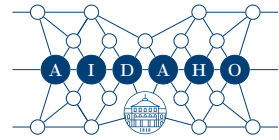
For this take home assignment, we have deployed a PostgreSQL database on the AIDAHO servers to store vessel data from the Automatic Identification System (AIS). AIS is a mandatory tracking system used by ships above a certain size to broadcast important information (position, speed, identity,...) in real time.

Each vessel has exactly one row in `ais_static` with fields such as IMO number, call sign, and draught, while the dynamic table `ais_dynamic` stores time-series position updates with geocoordinates, speed, and course for those vessels.

Within PSQL, the TimescaleDB extension is used to convert the dynamic AIS table into a time-partitioned hypertable. The data model is defined in the `.sql` file below.

— 1) Static vessel metadata (one row per MMSI)

```
CREATE TABLE IF NOT EXISTS ais_static (
  mmsi          INTEGER          PRIMARY KEY,
  imo           INTEGER,
  name          TEXT,
  call_sign     TEXT,
  flag          TEXT,
  draught       REAL,
  ship_type_code SMALLINT,
  ship_type     TEXT,
  length        REAL,
  width         REAL,
  eta           TIMESTAMP WITH TIME ZONE,
  destination   TEXT,
  static_updated_at TIMESTAMP WITH TIME ZONE
);
```



— 2) Dynamic AIS messages as a Timescale hypertable

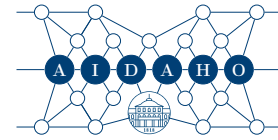
```
CREATE TABLE IF NOT EXISTS ais_dynamic (  
  created_at          TIMESTAMP WITH TIME ZONE, — original  
    ingestion timestamp  
  msg_timestamp       TIMESTAMP WITH TIME ZONE NOT NULL, — AIS  
    message time  
  position_updated_at TIMESTAMP WITH TIME ZONE,  
  mmsi                INTEGER NOT NULL,  
  latitude             DOUBLE PRECISION,  
  longitude            DOUBLE PRECISION,  
  speed               REAL, — SOG  
  course              REAL, — COG  
  heading             SMALLINT,  
  status              SMALLINT, — navigational status  
  maneuver             SMALLINT,  
  accuracy            BOOLEAN,  
  rot                 SMALLINT, — rate of turn  
  collection_type      TEXT,  
  geom                GEOGRAPHY(POINT, 4326) GENERATED ALWAYS AS (ST_  
    SetSRID(ST_MakePoint(longitude, latitude), 4326)) STORED,  
  PRIMARY KEY (msg_timestamp, mmsi)  
);  
— Convert the dynamic table into a hypertable on its message  
  timestamp  
SELECT create_hypertable(  
  'ais_dynamic',  
  'msg_timestamp',  
  if_not_exists => TRUE  
);  
— index on message time alone (helps where—clause on timestamp)  
CREATE INDEX ON ais_dynamic (msg_timestamp);  
— index on mmsi alone (helps lookups per ship)  
CREATE INDEX ON ais_dynamic (mmsi);  
— composite if you always filter both together  
CREATE INDEX ON ais_dynamic (mmsi, msg_timestamp);
```

Introduction to Data Science with R and RStudio

Dr. Johannes Bleher & Jan Dvorak

AIDAHO – AI & Data Science Certificate Hohenheim

Computational Science Hub



The AIS database is populated through an automated data ingestion pipeline that processes CSV files from two data providers, normalising their differing column schemas automatically. Dynamic position data is inserted in parallel using concurrent workers with bulk inserts, while static vessel metadata is deduplicated globally and inserted sequentially to avoid write conflicts. Indexes are dropped during ingestion and rebuilt afterwards to maximise throughput.

For the first parts of the assignment, you will mainly work with two API-exposed tables:

- **ais_static** – One row per vessel (**mmsi**), containing name, flag, ship type, call sign, and physical dimensions.
- **ais_dynamic** – One row per AIS position broadcast, containing timestamp, coordinates, speed, course, and heading. Implemented as a TimescaleDB hypertable partitioned by **msg_timestamp** for efficient time-range queries. A PostGIS **geom** column is computed automatically from latitude and longitude for spatial queries.

Both tables are linked by **mmsi** (Maritime Mobile Service Identity).

Additional task-specific tables, such as **ais_germany**, are introduced later where needed.

Important note: Access to the data is provided through PostgREST via the endpoint <https://aidaho-edu.uni-hohenheim.de/aisdb/>. PostgREST exposes database tables as REST resources and allows you to formulate queries through URL parameters, for example to select columns, filter rows, limit the number of returned records, and compute supported aggregations.

A single request may return at most 1,000,000 rows and will be automatically interrupted after 5 minutes. These limits protect the shared server from expensive unfiltered queries against large tables. In particular, do not issue unfiltered requests against **ais_dynamic**. Always restrict dynamic AIS queries by time interval, vessel identifier, aggregation, or **limit**.

```
GET /<table_name>?limit=5
```

```
GET /<table_name>?  
  select=<col1>,<col2>,<col3>  
  &<time_variable>=gte.2024-01-24T00:00:00Z  
  &limit=100
```

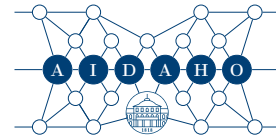
where **<table_name>**, **<col1>**, **<col2>**, **<col3>**, and **<time_variable>** are placeholders.

Introduction to Data Science with R and RStudio

Dr. Johannes Bleher & Jan Dvorak

AIDAHO – AI & Data Science Certificate Hohenheim

Computational Science Hub



For example:

```
GET /ais_static?limit=5
```

```
GET /ais_dynamic?
```

```
  select=msgsi,msg_timestamp,latitude,longitude,speed
  &msg_timestamp=gte.2024-01-24T00:00:00Z
  &limit=100
```

The PostgREST syntax is documented [here](#). You may also use an AI assistant to help interpret the documentation or construct queries, but you remain responsible for validating that the generated queries are correct, efficient, and do not overload the shared database.

This infrastructure provides access to a real geo-temporal AIS dataset stored in a PostgreSQL database with modern extensions for time-series and spatial data. The corresponding ingestion and database setup are provided in this [GitLab repository](#).

Task 1: Initiate a new Git repository - 5 points weight

- Create a new R project and initialize a local `git` repository. Push this repository to the AIDAHO GitLab.

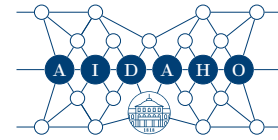
Name the repository `AIDAHO_IDS_THAS_2026` and use `main` as the default branch. Only the contents of the `main` branch will be considered for grading.

Add all team members as well as Johannes Bleher (`jbleher`) and Jan Dvorak (`jdvorak`) to the repository. Give Johannes Bleher and Jan Dvorak at least the role *Maintainer*. All team members should have permissions that allow them to push to the repository.

- Create the following folder structure in your repository:

```
AIDAHO_IDS_THAS_2026
├── 00_docs
├── 01_data
├── 02_code
├── 03_report
│   └── graphs
```

- Store this assignment sheet in `00_docs`. Your final report must be stored in `03_report`. All figures used in the report must also be saved separately in `03_report/graphs`.



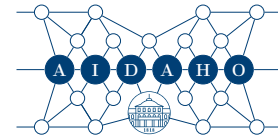
Task 2: Get an Overview - 15 points weight

In this task, you will get hands-on experience querying the AIS database through its PostgREST API. You will identify the available API resources, inspect example records, and compute basic summary statistics. Use R to send requests to the API and to process the returned results.

Be careful when querying large tables. In particular, do not download large parts of `ais_dynamic` into R. Whenever possible, use filtering, column selection, limits, and PostgREST aggregation so that the database performs the expensive work and R only receives the condensed result.

Unfiltered or weakly filtered queries against `ais_dynamic` can affect the performance of the shared database. Always restrict such queries by time interval, vessel identifier, aggregation, or `limit`.

1. In your report, start with a short introduction to AIS data. Explain how AIS data are collected and what information they typically contain. You may use online sources for this. This document provides a useful introduction. Cite all sources that you use.
2. Identify the available resources exposed by the PostgREST API. Which tables or views can you query through the API?
3. Provide a short overview of the data in your report. Address the following points:
 - (a) Briefly describe the AIS data model. What information is stored in the respective tables, and what do the most important fields represent?
 - (b) How many rows does each table contain? How many distinct vessels are recorded?
 - (c) Which vessel types are present in the data? Which vessel types occur most frequently? Discuss why these vessel types may dominate the dataset.
 - (d) Under how many distinct flags do the vessels operate? Which flag occurs most frequently?
Hint: Have a look at the PostgREST documentation on automatic group-by queries.
 - (e) Provide an overview of the columns in `ais_static`. What are their data types? For numeric columns, report suitable summary statistics such as minimum, maximum, average, and the number of non-missing observations where appropriate. For categorical columns, report the most frequent values and discuss visible patterns.



Note: PostgREST supports the aggregate functions `avg()`, `count()`, `max()`, `min()`, and `sum()`.

4. Use PostgREST filtering via URL query parameters to retrieve specific information from the database. Use combinations of filters where necessary. Report the query logic you used, not only the final answer.
 - (a) How many ships from Benelux countries are included in the dataset?
 - (b) How many German cargo ships are included in the dataset? How many of these are longer than 150 metres?
 - (c) How many vessels have **EXPRESS** in their name?
 - (d) How many dynamic AIS records exist for the first five minutes of 2022? How many distinct vessels are included in this five-minute snapshot?
 - (e) During the 30-minute interval from 2021-05-04 14:00:00 UTC to 2021-05-04 14:30:00 UTC, how many vessels were moving faster than 12 knots? How many of these vessels were cargo vessels?
 - (f) How can you limit the number of records returned by a PostgREST request? Demonstrate this with a small example query.

Task 3: Generating Sample Data - 20 points weight

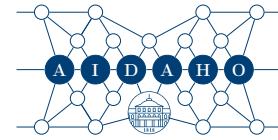
1. The `ais_dynamic` table contains hundreds of millions of high-frequency AIS observations and is therefore too large for interactive exploratory analysis. Before selecting a convenient subset of the data, it is important to think carefully about what constitutes a useful sample and what can go wrong during sampling.
 - (a) What is a sampling strategy? Define the term in your own words. Why is sampling necessary in data science, and why is it not simply the act of using less data? What properties should a good sample have relative to the population from which it is drawn?
 - (b) What does it mean for a sample to be representative? Name three dimensions along which a sample of AIS records could be non-representative and briefly explain the distortion each would introduce.
 - (c) Define availability bias in the context of data collection. Give an example of how availability bias can appear in AIS data. In your answer, consider the differences between terrestrial and satellite AIS receivers. What kinds of vessels, regions, or movement patterns might be systematically over- or under-represented?

Introduction to Data Science with R and RStudio

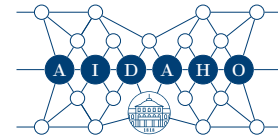
Dr. Johannes Bleher & Jan Dvorak

AIDAHO – AI & Data Science Certificate Hohenheim

Computational Science Hub



2. Create a script named `ais_dynamic_sample_points.R` in your `02_code` folder. Follow good programming practices: structure your code clearly, avoid unnecessary repetition, use meaningful object and function names, and add short comments where they help readability. If useful, create a separate functions folder and source your own helper functions from there.
 - (a) Generate all possible 5-minute intervals for January 24, 2024. These intervals form your sampling frame. Randomly select 100 intervals and retrieve at most 100 AIS observations from each selected interval, stopping once you have accumulated 10,000 observations in total. Ensure reproducibility by setting a random seed and explain in your report why this matters.
 - (b) Save the resulting sample as `sample_intervals.csv` in your `01_data` folder.
 - (c) This sampling approach is a form of cluster sampling, where the selected 5-minute intervals act as clusters. Identify at least two biases this approach may introduce and propose an alternative sampling strategy that would reduce the most serious of these biases.
3. Next, draw a second sample using a different strategy and compare it against your first sample.
 - (a) Query the API to retrieve the counts for all unique combinations of `ship_type` and hour of day present in the data for January 24, 2024. You may need to divide the day into several time intervals to avoid exceeding API response-size or runtime limits. The combinations of `ship_type` and hour of day form your strata.
 - (b) For each stratum, calculate its proportion of the total record count. Use these proportions to allocate a total sample size of 10,000 observations across strata. Larger strata should receive more observations. Make sure that very small strata are handled sensibly and document your allocation rule.
 - (c) Query each stratum separately and retrieve the allocated number of records. Where possible, retrieve a reproducible random subset from each stratum. If true random retrieval through the API is not possible, document and justify the approximation you use.
 - (d) Save the resulting sample as `sample_stratified.csv` in your `01_data` folder.
 - (e) For each of the variables `ship_type`, `speed`, and `collection_type`, produce a plot comparing the distribution in your interval sample and your stratified sample.



- (f) Compare the plots and identify where the two samples diverge. Explain how these differences arise from the way each sample was constructed.

Task 4: Vessel Behaviour - 20 points weight

1. Create a script named `sample_dashboard.R` in your `02_code` folder. The script should read the sampled AIS points from Task 3.2, stored in `01_data/sample_intervals.csv`, and display them on an interactive map using `leaflet`. If you were not able to complete Task 3.2, use the file `sample.csv` from the provided repository instead.

Make the map visually clear and suitable for inclusion in your report. Save the map or a screenshot of it and integrate it into your report.

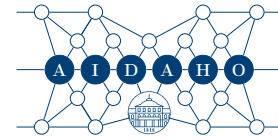
- (a) Indicate vessel speed using an appropriate colour scale. Add a legend and explain how the colours should be interpreted.
 - (b) Create a second version of the map that only uses observations with `collection_type = "satellite"`. Compare it with the full map. Does the spatial pattern change? Explain why this could be the case.
2. Write a script named `ais_dynamic_individual_paths.R` in your `02_code` folder. Use it to query, process, and visualize AIS tracks of individual vessels with `leaflet`. Follow good programming practices: use clear object names, avoid repeated code where possible, and define helper functions for repeated query or plotting logic.

When visualizing many AIS points in `leaflet`, use

```
clusterOptions = markerClusterOptions()
```

or another suitable strategy to avoid rendering too many points at once.

- (a) Inspect the data for `MMSI = 2579999`. Do you suspect any data quality issues? Search online for information about this MMSI and briefly discuss how such data may have been generated. What might be happening here?
- (b) Examine the movement of the vessel with `MMSI = 563040400`. Query a suitable time window, visualize the vessel's track, and describe its behaviour during the observed period.
- (c) Analyze the movement of the vessel with `MMSI = 211430830` over the time window from `2021-01-17 00:00:00 UTC` to `2021-01-20 00:00:00 UTC`. Along the route between Vienna and Linz, the vessel passes multiple ship locks, where vessels are raised or lowered to bypass elevation differences or infrastructure obstacles such as dams. Describe how a ship lock typically appears in AIS data. In your answer, consider speed, location, duration, and the density of AIS observations in the same area.



- (d) Write a function that automatically detects potential ship-lock events for the vessel and time window from task c).

Your function may use a rule-based heuristic. For example, you can proceed as follows:

- i. Sort the AIS observations by `msg_timestamp`.
- ii. Mark observations as potential lock points if the vessel speed is below a chosen threshold, for example below 1 knot.
- iii. Group consecutive potential lock points if they are close to each other in space and time.
- iv. Remove groups that are too short to represent a plausible lock passage.
- v. For each remaining group, calculate the start time, end time, duration, approximate location, and number of AIS observations.

Consecutive points belonging to the same potential lock event should be assigned to the same lock passage. Your code should return a table of detected lock passages with at least the following columns: `start_time`, `end_time`, `duration_minutes`, `latitude`, `longitude`, and `n_observations`. Explain and justify the thresholds or rules used by your function. It is not necessary that your algorithm detects every lock perfectly; the goal is to develop a transparent, reproducible detection strategy. Discuss its limitations and how you could improve it further. Represent your rule-based detection logic as a small decision tree, flowchart, or ordered list of decisions.

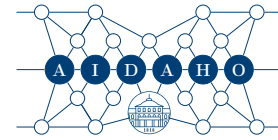
Optional extension: Discuss how an unsupervised approach could be used instead of manually labelled training data.

- (e) Plot the vessel's trajectory using `leaflet`. Your visualization should include the route of the vessel and the locations of the detected ship-lock passages. Use a clear visual distinction between regular trajectory points and detected lock events.

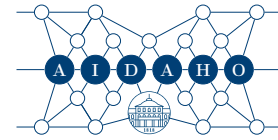
Task 5: AIS Traffic Density - 20 points weight

1. In this task, you will combine AIS data with geographic information about German rivers to analyse how vessels move through the inland waterway network. We provide a table called `ais_germany`, which contains AIS records for vessels tracked within Germany. Use this table for the following tasks.

- (a) Use the `rnaturalearth` package to retrieve river geometries for Germany in R. Inspect the data carefully, keep only relevant river features, and remove duplicate entries where necessary.



- (b) Generate a map of the German river geometries. Each river should be indicated by a different colour where feasible. Add an appropriate legend. If there are too many small rivers for a readable legend, focus the legend on the major rivers and explain your choice.
 - (c) Query the `ais_germany` table to retrieve all tracking records from 2022-04-23. Use appropriate filters so that only records from this date are returned.
2. To determine on which rivers vessels are travelling, you will use the H3 spatial indexing system. H3 divides the world into a hierarchical grid of hexagonal cells, each identified by a unique index. The resolution of the grid can be adjusted from 0 (very coarse, continent-scale cells) to 15 (very fine, metre-scale cells).
 - (a) Explain the general advantages of H3's hexagonal grid over naive spatial approaches, such as checking each AIS point against each river geometry or computing distances from each AIS point to all river segments. Describe the concept of hash tables, explain how H3 indices can be used as keys, and articulate why this can make spatial matching more efficient for large datasets.
 - (b) Calculate the H3 index for each AIS record retrieved in the previous step. Use the `latitude` and `longitude` columns for this. Also calculate the H3 indices that cover the German river geometries. Since rivers are represented as line geometries, apply an appropriate buffer around the river geometries before generating the river H3 cells. Choose H3 resolution 7 for this task. Discuss the selected buffer distance in your report.
 - (c) Match the H3 indices of the AIS records to the H3 indices of the buffered river geometries to determine whether each AIS record is located on a German river. If an AIS record matches more than one river, define and document a rule for assigning it to one river, for example by choosing the nearest river geometry.
 - (d) Calculate the percentage of AIS records on 2022-04-23 that were recorded on a German river versus elsewhere, such as coastal or open-water areas.
 - (e) Create a grouped bar chart comparing the distribution of ship types between vessels tracked on rivers and vessels tracked elsewhere.
 - (f) Return a summary table showing the name of each river and the number of AIS records associated with it. Sort the table in descending order by the number of records.
 - (g) Create a plot showing the distribution of vessel traffic along the Rhine. To do this, keep only AIS records associated with the Rhine, project each point onto the Rhine geometry, calculate its approximate distance along the river, and



aggregate records into distance intervals, for example 10 km bins. The x-axis should show the distance along the Rhine in kilometres, and the y-axis should show the density of AIS records in each interval. Discuss any assumptions you make about the start point and direction of the Rhine geometry.

Task 6: Static Dashboard - 7.5 points weight

For the following tasks, integrate the files hosted in the GitLab repository

https://aidaho-edu.uni-hohenheim.de/gitlab/jdvorak/shiny_thas_2026

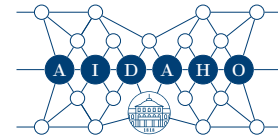
into your own THAS repository. The repository contains a `docker-compose.yaml` file that sets up an NGINX server and a Shiny app. The Shiny app will be used later in Task 7. In this task, you will first create and serve a static HTML dashboard.

1. In your `02_code` folder, create a script named `sample_html_dashboard.R`. Modify your code from Task 4.1 so that the generated `leaflet` map is saved as a self-contained HTML file named `sample_points.html` using `htmlwidgets::saveWidget()`.
2. Store `sample_points.html` in a location that is mounted into the document root of the NGINX server. Adjust the relevant configuration or volume mapping if necessary.
3. Test the setup locally with Docker Compose. Verify that the static dashboard can be opened in a browser before deploying it to the server.
4. After local testing, clone or update your repository on your server and start the Docker Compose setup there.
5. Test and debug your system so that the static dashboard is served under `https://<your-server>/sample_points.html`. In your report, briefly document your setup and include a screenshot or link showing that the deployed page works.

Task 7: R Shiny Application - 12.5 points weight

In the final part of this assignment, you will build and deploy a Shiny application. The repository introduced in Task 6 already contains a Shiny app skeleton. Your task is to fix the existing problems, connect the app to the Docker setup, and implement the application logic.

1. Modify the `docker-compose.yaml` file so that the Shiny app files are mounted into the Shiny container. Make sure that changes to the files in the `/shiny` folder are available inside the container.



2. In the `/shiny` folder, you will find the skeleton of a Shiny application. Modify the relevant files to implement one of the following two options. The files contain hints on what needs to be changed.

Depending on whether you completed Task 5, choose one of the following options:

(a) **Option A – If you completed Task 5:**

Reuse and extend your logic from Task 5.2(g) to make the traffic-density plot interactive. Users should be able to select a `river_name` and one or more `ship_type` values. After making their selections, users should click a `refresh` button to update the plot. The plot should show the traffic density on the selected river for the selected ship types on 2022-04-23; see Task 5.2(g) for details.

(b) **Option B – If you did not complete Task 5:**

Create an interactive plot showing the distribution of AIS records within a one-hour period on 2022-04-23. Users should be able to select a one-hour interval, a `collection_type`, and one or more `ship_type` values. After making their selections, users should click a `refresh` button to update the plot. The plot should show the minute of the selected hour on the x-axis, from 0 to 59, and the number of AIS records on the y-axis.

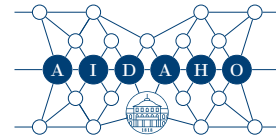
Optional: Group the data by `ship_type` and `collection_type`, for example terrestrial versus satellite, and use colour, facets, or another suitable visual encoding to distinguish the groups.

3. Avoid repeatedly sending expensive API requests from the deployed Shiny app. Where possible, preprocess the required data in an R script and let the Shiny app read the prepared data file.
4. Adjust the NGINX configuration so that the Shiny app is accessible via the endpoint `/ais_app`.
5. Test the Shiny app locally with Docker Compose. Verify that the interface loads, the inputs work, and the plot updates when the `refresh` button is clicked.
6. After local testing, clone or update your repository on your server and start the Docker Compose setup there. Test and debug your system so that the Shiny app is served under `https://<your-server>/ais_app`. In your report, briefly document your setup and include a screenshot or link showing that the deployed app works.

Introduction to Data Science with R and RStudio

Dr. Johannes Bleher & Jan Dvorak

AIDAHO – AI & Data Science Certificate Hohenheim
Computational Science Hub



This assignment is due on June 19, 2026, end-of-day. Please hand in your report via ILIAS and save it in your AIDAHO GitLab repository. Assignments without a duly submitted report are not graded. Reference all team members with matriculation numbers in your report. Also reference your AIDAHO GitLab repository.

For the overall grade, the task-specific components are weighted with 100 points. Compliance with the general THAS guidelines is weighted with an additional 30 points. The maximum score is therefore 130 points.

The 30 guideline points are awarded for general quality and compliance with the THAS guidelines:

- *code quality, reproducibility, and user-friendliness: 10 points,*
- *report structure, interpretation, language, and concision: 10 points,*
- *graph and table quality, repository structure, README, citations, and documentation of AI use: 10 points.*

The pass threshold is at least 65 out of 130 points.

If you fail the assignment, you may submit a revised version by June 30, 2026. For revised submissions after the regular deadline, the maximum achievable score is limited to 75% of the maximum THAS score. This rule applies only to revised submissions after a failed first submission, not to valid deadline extensions granted due to illness or other accepted reasons.

To pass, students must submit at least:

- *a written report as a .pdf file,*
- *a GitLab repository with the required folder structure,*
- *executable R scripts for at least two of the three major analytical tasks worth 20 points,*
- *at least one generated sample dataset,*
- *evidence that the static dashboard or Shiny component was attempted.*

The main report should be concise and is limited to seven pages, excluding the title page, references, and appendices.

If you are unable to meet the deadline due to illness or another valid reason, please contact us before June 19, 2026.